

Resultados y Hallazgos:

- Registros de Interacción con IA:

Para potencia promedio: “Necesito el código para un bloque 'Embedded Python Block' de GNU Radio. El bloque debe tener una entrada un vector de 1024 de longitud de entradas complejas y una salida única tipo float. Esta salida será la potencia promedio de un vector $x[n]$. La función work debe calcular la potencia promedio de la ventana de entrada. La potencia se define como la media del cuadrado de la magnitud de las muestras. Asegúrese de importar la biblioteca numpy.”

Para estadísticas de amplitud: “Ahora generaremos el bloque 2, que es Estadísticas de Amplitud. Genere el código para un 'Embedded Python Block' de GNU Radio con una entrada (vector, tipo complex) y dos salidas (un solo ítem por salida, ambas tipo float). La salida `out[0]` debe ser la media de la magnitud de las muestras de entrada (calculada como `numpy.mean(numpy.abs(in_sig[0]))`). La salida `out[1]` debe ser la desviación estándar de la magnitud de esas mismas muestras.”

- Código generado y corregido:

Para potencia promedio:

```
1  import numpy as np
2  from gnuradio import gr
3
4  class blk(gr.sync_block):
5      def __init__(self):
6          gr.sync_block.__init__(
7              self,
8              name='Potencia promedio',
9              in_sig=[(np.complex64, 512)],
10             out_sig=[np.float32]
11         )
12
13  def work(self, input_items, output_items):
14      x = input_items[0] # x tiene forma [N, 1024]
15      N = 1024
16
17      # Potencia promedio explícita:  $(1/N) * \sum(|x[n]|^2)$ 
18      power = (1.0 / N) * np.sum(np.abs(x)**2, axis=1)
19
20      output_items[0][:] = power.astype(np.float32)
21      return len(output_items[0])
```

Para estadísticas de amplitud:

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block):
    """Bloque para calcular media y desviación estándar de la amplitud"""

    def __init__(self):
        gr.sync_block.__init__(
            self,
            name="Estadísticas de Amplitud",
            in_sig=[(np.complex64, 512)], # entrada: vector complejo (longitud configurable)
            out_sig=[np.float32, np.float32] # salidas: media y desviación estándar
        )

    def work(self, input_items, output_items):
        in0 = input_items[0] # forma: (N, 1024)
        mean_out = output_items[0]
        std_out = output_items[1]

        for i, vec in enumerate(in0):
            # Calcular magnitudes
            magnitudes = np.abs(vec)

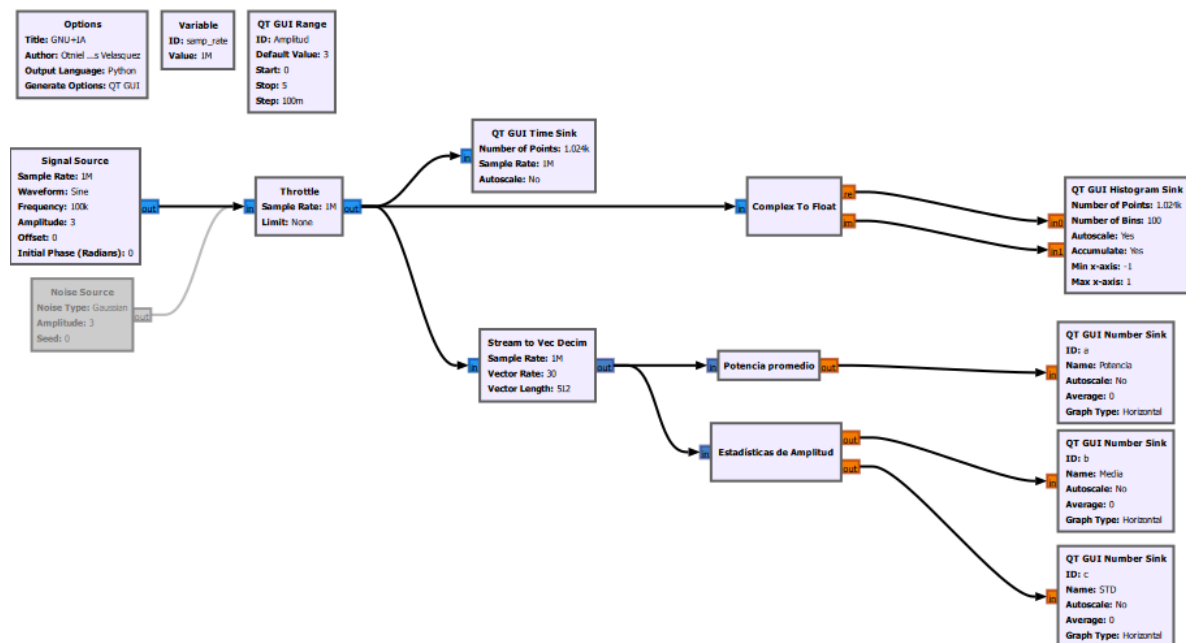
            # Media de amplitud
            mean_amp = np.mean(magnitudes)

            # Desviación estándar de amplitud
            std_amp = np.std(magnitudes)

            # Asignar resultados a las salidas
            mean_out[i] = mean_amp
            std_out[i] = std_amp

        # Retornar el número de elementos producidos
        return len(mean_out)
```

- Diagrama de Flujo:



- Tabla de validación experimental:

Fuente de Señal	Parámetro	Valor Teórico	Valor Medido
Ruido (Amp = 1)	Potencia	0.709	0.485792
	Media	0.886	0.867242
	STD	0.463	0.468482
Ruido (Amp = 2)	Potencia	1.418	2.141261
	Media	1.772	1.821031
	STD	0.926	0.983040
Ruido (Amp = 3)	Potencia	2.127	4.763582
	Media	2.658	2.749418
	STD	1.389	1.402805
Sinusoidal (Amp = 1)	Potencia	0.5	0.499997
	Media	1	0.999997
	STD	0	0.000001
Sinusoidal (Amp = 2)	Potencia	2	1.999998
	Media	2	1.999994
	STD	0	0.000003
Sinusoidal (Amp = 3)	Potencia	4.5	4.499971
	Media	3	2.999990
	STD	0	0.000004